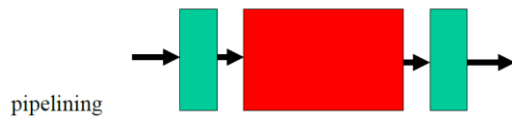


DATA FLOW MODELING

Any digital system can be constructed by interconnecting registers and adding combinational logic between them to perform the necessary functions.

- Dataflow provides a powerful way to implement a design.
- Logic synthesis tools can be used to create a gate-level circuit from a dataflow design description.
- RTL (register transfer level) is a combination of dataflow and behavioral modeling.

Describes how data moves from inputs to outputs using Boolean equations and operators.



ANATOMY OF AN ASSIGNMENT

At a fundamental level in Verilog, an assignment consists of two parts: a **Left-Hand Side (LHS)** and a **Right-Hand Side (RHS)** separated by = or <=.

- **RHS:** This is any expression that evaluates to a value to which the LHS value is to be assigned.
- **LHS:** This is the target that receives the value, and it can take specific forms depending on the type of assignment (e.g., a wire in continuous assignments, or a reg in procedural ones).

ESSENTIAL OPERATORS

Arithmetic	Bitwise	Reduction	Relational
+: add -: subtract *: multiply /: divide %: modulus **: exponent	~ : NOT & : AND : OR ^ : XOR ~^, ^~ : XNOR	& : AND : OR ~& : NAND ~ : NOR ^ : XOR ~^, ^~ : XNOR	>: greater than <: less than >=: greater than or equal <=: less than or equal
Shift	case equality ===: equality !==: inequality	Logical &&: AND : OR !: NOT	Miscellaneous {, }: concatenation {c { }}: replication ?: conditional
<<: left shift >>: right shift <<<: arithmetic left shift >>>: arithmetic right shift	Equality ==: equality !=: inequality		

Operator Type	Symbols	Examples
Bitwise	~ (NOT), & (AND), (OR), ^ (XOR)	assign Y = A & B;
Logical	! (NOT), && (AND), (OR)	assign is_valid = (A && B);
Arithmetic	+, -, *, /, %	assign Sum = A + B;
Relational	>, <, >=, <=, ==, !=	assign is_greater = (A > B);
Conditional	? : (Ternary operator)	assign Y = Sel ? A : B;
Concatenation	{ }	assign Bus = {A, B};

TWO BASIC FORMS OF ASSIGNMENTS

Continuous assignment: assign values to nets

Procedural assignment: assign values to variables

Statement type	Left-hand side
Continuous assignment	Net (vector or scalar) Constant bit-select of a vector net Constant part-select of a vector net Constant indexed part-select of a vector net Concatenation or nested concatenation of any of the above left-hand side
Procedural assignment	Variables (vector or scalar) Bit-select of a vector reg, integer, or time variable Constant part-select of a vector reg, integer, or time variable Indexed part-select of a vector reg, integer, or time variable Memory word Concatenation or nested concatenation of any of the above left-hand side

STANDARD PROCEDURAL BLOCKS

initial Block: Starts at time 0 and executes sequentially exactly once. Used for testbench stimulus.

always Block: Runs in an infinite loop, triggered by a sensitivity list.

Crucial Rule: Any LHS variable assigned a value inside an initial or always block must be a reg. Any LHS variable assigned outside via an assign statement must be a wire.

PROCEDURAL CONTINUOUS ASSIGNMENTS

While **assign** is normally used outside procedural blocks for standard data flow, there are two additional forms of assignments known as procedural continuous assignments. These are advanced simulation constructs used inside procedural blocks (like initial blocks in testbenches) to dynamically override the normal behavior of a circuit: **assign / deassign** and **force / release**

assign / deassign: Used to force a specific value onto a reg variable, overriding standard procedural assignments.

force / release: A stronger version that can override both reg variables and wire nets. This is incredibly useful in testbenches for injecting faults or forcing a specific internal node to a known state without rewriting the core module logic.

A continuous assignment begins with the keyword **assign**.

`assign net_lvalue = expression;`

`assign net1 = expr1, net2 = expr2, ..., netn = exprn;`

Assignments on the nets shall be **continuous** and **automatic**.

This means that:

- whenever an operand in the RHS expression changes value, the whole RHS shall be evaluated, and if the new value is different from the previous value, then the new value shall be assigned to the LHS.

An **implicit continuous assignment:**

- is the shortcut of declaring a net first and then writing a continuous assignment on the net.
- is always active.

- can only have one implicit declaration assignment per net.

```

wire out;           // net declaration
assign out = in1 & in2; // regular continuous assignment wire

out = in1 & in2; // implicit continuous assignment

```

CALCULATING AND SIMULATING DELAYS

In physical hardware, logic gates are not instantaneous. Verilog allows us to model real-world physics using delay values.

PROPAGATION DELAY

Propagation delay is the time it takes for a change at the input to produce a change at the output.

Syntax: assign #delay_value target = expression;

Example: assign #5 Y = A & B;

(If A or B changes, the simulator waits 5 time units before updating Y).

INERTIAL DELAY (GLITCH FILTERING)

Continuous assignments inherently model inertial delay. Physical gates have a certain "inertia"—they require an input to be held steady for a minimum amount of time to actually flip the output state.

The Rule: If you define **assign #5 Y = A & B;**, and A pulses high for only 3 time units, the output Y will not change. The simulator filters this out as a "glitch."

CALCULATING AND SIMULATING DELAYS

Three ways of specifying delays:

- Regular assignment delay
- Implicit continuous assignment delay
- Net declaration delay

REGULAR DELAYS

The delay value is specified after the keyword assign.

The inertial delay model is used (default model).

Example:

```

wire in1, in2, out;

assign #10 out = in1 & in2;

```

CONTINUOUS ASSIGNMENT AND DELAYS

In situations where an RHS operand changes before a previous change has had to propagate to the LHS, then the following steps are taken.

1. The value of the RHS expression is evaluated
2. If the RHS value **differs** from the value currently scheduled to propagate the LHS, then the currently scheduled propagation event is **descheduled**.
3. If the new RHS value **equals** the current LHS value, **no event is scheduled**.

In situations where an RHS operand changes before a previous change has had to propagate to the LHS, then the following steps are taken.

4. If the new RHS value differs from the current LHS value, a delay is calculated in the standard way **using the current value of the LHS, the newly calculated value of the RHS, and the delays indicated on the statement**; a new propagate event is then scheduled to occur delay time units in the future.

NET DECLARATION DELAYS

A net can be declared associated with a delay value.

Net declaration delays can also be used in gate-level modeling.

```

// net delays wire
#10 out;           // transport delay

assign out = in1 & in2;

// regular assignment delay
wire out;
assign #10 out = in1 & in2;
// inertial delay

```

THE BASIS OF DATAFLOW MODELING

The essence of dataflow modeling is:

expression = operators + operands

- **Operands** can be any one of the allowed data types.
- **Operators** act on the operands to produce desired results.

Operands:

- constants
- integers
- real numbers
- nets
- registers
- time
- bit-select
- part-select
- memories
- function calls

PRECEDENCE OF OPERATORS

Operators	Symbols	Precedence
Unary	+ - ! ~	Highest
Exponent	**	↑
Multiply, divide, modulus	* / %	
Add, subtract	+ -	
Shift	<< >> <<< >>>	
Relational	< <= > >=	
Equality	== != === !==	
Reduction	& ~& & ^ ~^ ~	
Logical	&& 	
Conditional	?:	Lowest